

Курсовая работа

Пападык Всеволод Дмитриевич

МИФИ

Построить прогноз, позволяющий подобрать наиболее эффективное сочетание параметров
для создания лекарственных препаратов

Введение

Постановка проблемы

Химиками были предоставлены данные о 1001 химическом соединении с указанием их эффективности против вируса гриппа. Параметры, характеризующие эффективность, обозначаются как IC50, CC50 и SI. Нужно спрогнозировать эффективность соединений и подобрать наиболее подходящие сочетания параметров для разработки лекарственных средств.

Актуальность темы

Существует бесчисленное множество химических соединений. Проверять каждое из них долго и дорого. Применение алгоритмов машинного обучения может помочь сузить это множество. Что в свою очередь увеличит вероятность нахождения эффективного лекарства. Как следствие, лекарства могут стать дешевле (по этому направлению главный фактор это патентная система США, но в России такой проблемы нет), эффективнее. Быстрее будет поиск лекарств от болезней с которыми человечество еще не научилось бороться. Поэтому применение машинного обучения крайне актуально в решении химических задач.

Цель работы

Необходимо построить модели регрессии и классификации медианного значения для параметров IC50, CC50 и SI. Дополнительно можно выделить наиболее значимые признаки для данных таргетов.

Задачи работы

Чтобы решить эту задачу необходимо проанализировать данный датасет: выяснить какие признаки представлены и что они означают. Обработать пропуски, выбросы.

Проанализировать данные статистическими методами. Обучить модели и сравнить их между

собой. Подобрать метрики, по которым будет осуществляться проверка эффективности моделей. Настроить оптимальные гиперпараметры для лучшей модели.

Методы исследования

Будет использован язык программирования python с библиотеками numpy, pandas, sklearn, scipy и другие. Платформа jupyter notebook.

Методология

Описание данных

Таргеты:

1. **IC50 (полумаксимальная ингибирующая концентрация)**: это концентрация вещества, необходимая для ингибирования определенного биологического или биохимического процесса на 50%. Чем ниже значение IC50, тем более эффективным считается соединение в ингибировании целевого процесса.
2. **CC50 (полумаксимальная цитотоксическая концентрация)**: это концентрация вещества, при которой 50% клеток погибают или теряют свою функцию. Это значение используется для оценки токсичности соединения. Более высокое значение CC50 указывает на меньшую токсичность.
3. **SI (индекс селективности)**: это отношение CC50 к IC50 ($SI = CC50 / IC50$). Индекс селективности показывает, насколько избирательно соединение действует на вирусные клетки по сравнению с нормальными клетками. Более высокое значение SI указывает на более высокую селективность и потенциально более безопасное соединение для использования.

Таким образом, если вещество имеет низкое значение IC50 и высокое значение CC50, это значит, что оно эффективно против вируса и в то же время безопасно для нормальных клеток

Признаки:

1. **MaxAbsEStateIndex, MaxEStateIndex, MinAbsEStateIndex, MinEStateIndex**: Эти параметры связаны с электронным состоянием атомов в молекуле. Они описывают максимальные и минимальные значения индексов электронного состояния, которые могут быть абсолютными или относительными.

2. **qed (Quantitative Estimate of Drug-likeness)**: Это количественная оценка "лекарственности" соединения, основанная на различных химических свойствах.

3. **SPS (Synthetic Accessibility Score)**: Оценка синтетической доступности соединения, то есть насколько легко или сложно его синтезировать.

4. **MolWt, HeavyAtomMolWt, ExactMolWt**: Молекулярный вес соединения. HeavyAtomMolWt учитывает только тяжелые атомы (не водород), а ExactMolWt — точный молекулярный вес.

5. **NumValenceElectrons, NumRadicalElectrons**: Количество валентных электронов и радикальных электронов в молекуле.

6. **MaxPartialCharge, MinPartialCharge, MaxAbsPartialCharge, MinAbsPartialCharge**: Эти параметры описывают максимальные и минимальные частичные заряды атомов в молекуле.

7. **FpDensityMorgan1, FpDensityMorgan2, FpDensityMorgan3**: Плотность отпечатков (fingerprints) Morgan на различных радиусах. Эти параметры используются для описания структуры молекулы.

8.**BCUT2D_MWHI, BCUT2D_MWLOW, BCUT2D_CHGHI, BCUT2D_CHGLO, BCUT2D_LOGPHI, BCUT2D_LOGPLOW, BCUT2D_MRHI, BCUT2D_MRLOW**: Это параметры, связанные с молекулярными дескрипторами BCUT, которые оценивают различные аспекты молекулярной структуры, такие как молекулярный вес, заряд, логарифм распределения и молекулярный рефрактивный индекс.

9.**Avglpc, BalabanJ, BertzCT**: Различные индексы, которые оценивают сложность и связность молекулы.

10.**Chi0, Chi0n, Chi0v, Chi1, Chi1n, Chi1v, Chi2n, Chi2v, Chi3n, Chi3v, Chi4n, Chi4v**: Индексы Хи (Chi indices), которые описывают различные аспекты топологии молекулы.

11.**HallKierAlpha**: Параметр, связанный с альфа-индексом Холла-Киера, который оценивает гибкость молекулы.

12.**Ipc**: Индекс информационной сложности молекулы.

13.**Kappa1, Kappa2, Kappa3**: Индексы формы молекулы, которые оценивают ее трехмерную структуру.

14.**LabuteASA**: Площадь доступной поверхности молекулы, рассчитанная по методу Лабута.

15.**PEOE_VSA1, PEOE_VSA2, ..., PEOE_VSA14**: Параметры, связанные с объемной поверхностью молекулы, рассчитанные с использованием метода PEOE (Partial Equalization of Orbital Electronegativities).

16.**SMR_VSA1, SMR_VSA2, ..., SMR_VSA10**: Параметры, связанные с объемной поверхностью молекулы, рассчитанные с использованием метода SMR (Simple Molecular Refractivity).

17. **SlogP_VSA1, SlogP_VSA2, ..., SlogP_VSA12**: Параметры, связанные с объемной поверхностью молекулы, рассчитанные с использованием метода SlogP (логарифм коэффициента распределения).
18. **TPSA**: Полярная поверхность молекулы, которая может влиять на ее способность проникать через клеточные мембраны.
19. **EState_VSA1, EState_VSA2, ..., EState_VSA11**: Параметры, связанные с объемной поверхностью молекулы, рассчитанные с использованием метода EState (Electronic State).
20. **VSA_EState1, VSA_EState2, ..., VSA_EState10**: Параметры, связанные с объемной поверхностью молекулы, рассчитанные с использованием метода VSA (Volume Surface Area) и EState.
21. **FractionCSP3**: Доля атомов углерода в молекуле, имеющих sp^3 -гибридизацию.
22. **HeavyAtomCount**: Количество тяжелых атомов (не водород) в молекуле.
23. **NHONCount, NOCount**: Количество функциональных групп, содержащих N-H или O-H связи, и количество атомов азота и кислорода.
24. **NumAliphaticCarbocycles, NumAliphaticHeterocycles, NumAliphaticRings**: Количество алифатических карбоциклов, гетероциклов и колец.
25. **NumAromaticCarbocycles, NumAromaticHeterocycles, NumAromaticRings**: Количество ароматических карбоциклов, гетероциклов и колец.
26. **NumHAcceptors, NumHDonors**: Количество акцепторов и доноров водородных связей.

27.**NumHeteroatoms**: Количество гетероатомов в молекуле.

28.**NumRotatableBonds**: Количество вращаемых связей в молекуле.

29.**NumSaturatedCarbocycles, NumSaturatedHeterocycles, NumSaturatedRings**: Количество насыщенных карбоциклов, гетероциклов и колец.

30.**RingCount**: Общее количество колец в молекуле.

31.**MolLogP**: Логарифм коэффициента распределения (гидрофобность) молекулы.

32.**MolMR**: Молекулярный рефрактивный индекс.

33.**fr_AI_COO, fr_AI_OH, ..., fr_urea**: Эти параметры описывают наличие различных функциональных групп в молекуле, таких как карбоксильные группы, гидроксильные группы, амины, эфиры и другие.

Эти дескрипторы используются для анализа и моделирования химических свойств соединений, а также для предсказания их биологической активности.

Предобработка данных

Был удален признак 'Unnamed: 0' поскольку он был порядковым номером. Хотя на удивление модели считали его самым значимым и после удаления метрики r^2 падали на 0.2-0.3, но закономерностей связанных с этим не было обнаружено. Но очевидно что он не может нести информативной нагрузки и поэтому может только запутать, особенно если захотим применить аналогичные методы к аналогичному датасету.

Очистка данных включает в себя обработку пропусков в данных - их было немного 3 записи из 1001. Они были заполнены медианными значениями.

	IC50, mM	CC50, mM	SI
78	1127.094988	1127.094988	1.000000
79	25.171788	1878.491646	74.626866
80	1199.174968	1199.174968	1.000000

Полностью идентичные IC50 и CC50 в 78 и 80 экземплярах. 79 экземпляр выглядит крайне эффективным, для него можно заполнить пропуски медианным значением. Для IC50 и SI можно удалить записи в которых IC50 и CC50 полностью совпадают - таких оказалось 135 записей.

Далее была проведена проверка линейных зависимостей в данных - их оказалось очень много. Будут удалены после агрегации данных.

Были удалены признаки, у которых одинаковые значения во всех экземплярах.

Использован Normalizer для нормализации данных. Бинаризация для fr-признаков.

Используя взаимодействие между признаками и нелинейные преобразования, добавлены новые признаки(агрегация).

Использовались методы фильтрации признаков: PCA, KBest, RFE,

sklearn.feature_selection.SelectFromModel, sklearn.feature_selection.SelectFromModel. f_classif. Хотя вынужден

признать они не дали особой пользы, хотя времени на них было потрачено больше, чем нужно.

Было построено распределение таргета и его логарифмированное значение.

Модели машинного обучения

Для регрессии использовались модели: LinearRegression, KNeighborsRegressor, RandomForestRegressor, DecisionTreeRegressor, KernelRidge, XGBRegressor, LGBMRegressor, CatBoostRegressor, GradientBoostingRegressor, HistGradientBoostingRegressor, AdaBoostRegressor.

Для классификации: LogisticRegression, KNeighborsClassifier, RandomForestClassifier, DecisionTreeClassifier, SVC, XGBClassifier, LGBMClassifier, CatBoostClassifier, GradientBoostingClassifier, HistGradientBoostingClassifier, AdaBoostClassifier, GaussianNB

Оценка моделей

для регрессии использовались метрики: R^2 , MAE

для классификации использовались метрики: roc auc, accuracy, precision, recall,

F1-score(последние три метрики weighted avg и macro avg)

Экспериментальная часть

Настройка экспериментов

Были созданы python-файлы для общих скриптов, общая часть для всех задач по обработке данных была в аналитическом ноутбуке с сохранением нового датасета.

Результаты регрессии

IC50

[19]:

	Model	Mean Absolute Error	R2 Score
2	Random Forest	109.03	0.51
9	Krr	107.57	0.48
5	LightGBM	109.54	0.46
6	CatBoost	108.72	0.46
7	HistGradientBoosting	109.83	0.45
4	Gradient Boosting	113.80	0.43
1	KNeighbors	110.94	0.42
3	XGBoost	109.60	0.42
8	AdaBoost	136.32	0.26
0	Decision Tree	130.37	0.10

лучшая модель с гиперпараметрами:

```
[136]: best_model = RandomForestRegressor(  
        criterion='poisson',  
        max_depth=16,  
        max_features='log2',  
        max_samples=0.97,  
        min_samples_leaf=1,  
        min_samples_split=4,  
        n_estimators=241,  
        random_state=42,  
    )  
  
    best_model.fit(X_train, y_train)  
    y_pred = best_model.predict(X_test)  
  
    mae = mean_absolute_error(y_test, y_pred)  
    r2 = r2_score(y_test, y_pred)  
  
    print(f'Mean Absolute Error: {mae:.3f}')  
    print(f'R2 Score: {r2:.3f}')
```

Mean Absolute Error: 106.946
R2 Score: 0.510

[12]:

	Model	Mean Absolute Error	R2 Score
9	Krr	272.88	0.48
4	Gradient Boosting	264.24	0.47
2	Random Forest	277.54	0.45
3	XGBoost	271.62	0.45
6	CatBoost	297.65	0.43
5	LightGBM	293.47	0.41
7	HistGradientBoosting	301.20	0.40
8	AdaBoost	395.13	0.26
0	Decision Tree	282.97	0.25
1	KNeighbors	305.79	0.15

лучшая модель с гиперпараметрами:

```

best_model = GradientBoostingRegressor(
    learning_rate=0.04,
    max_depth=7,
    max_features='log2',
    max_leaf_nodes=10,
    min_impurity_decrease=0.006,
    min_samples_leaf=5,
    min_samples_split=4,
    n_estimators=314,
    subsample=0.82,
    random_state=725,
    verbose=0,
)
best_model.fit(X_train[featureSelector.rfe_best_feature_names], y_train)
y_pred = best_model.predict(X_test[featureSelector.rfe_best_feature_names])

mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Absolute Error: {mae:.3f}')
print(f'R2 Score: {r2:.3f}')

Mean Absolute Error: 283.611
R2 Score: 0.445

```

SI

	Model	Mean Absolute Error	R2 Score
3	LightGBM	23.04	0.37
4	CatBoost	22.52	0.36
0	Random Forest	22.94	0.34
6	Krr	23.23	0.34
5	HistGradientBoosting	24.34	0.31
1	XGBoost	24.61	0.14
2	Gradient Boosting	24.58	0.09

лучшая модель с гиперпараметрами:

```
best_model = CatBoostRegressor(  
    bagging_temperature=0,  
    depth=8,  
    iterations=300,  
    l2_leaf_reg=3,  
    learning_rate=0.01,  
    verbose=0,  
    random_state=42,  
)  
  
best_model.fit(X_train[common_selected_features], y_train)  
y_pred = best_model.predict(X_test[common_selected_features])  
  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
  
print(f'Mean Absolute Error: {mae:.3f}')  
print(f'R2 Score: {r2:.3f}')
```

Mean Absolute Error: 21.199
R2 Score: 0.437

Наиболее значимые признаки:

'NumRotatableBonds', 'fr_Al_COO', 'fr_Al_OH_noTert', 'fr_C_O',
'fr_Imine', 'fr_NH1', 'fr_NH2', 'fr_alkyl_halide', 'fr_allylic_oxid',
'fr_aryl_methyl', 'fr_benzene', 'fr_bicyclic', 'fr_para_hydroxylation',
'fr_quatN', 'fr_sulfonamd', 'fr_unbrch_alkane', 'VSA_EState_median',
'VSA_EState_variation', 'PEOE_VSA_variation', 'SlogP_VSA_variation',
'Chi_variation', 'BCUT2D_variation', 'Saturation1', 'Flexibility',
'HeavyAtomFraction'

Результаты классификации

IC50 median

[14]:

	Model	WA f1	accuracy	WA precision	WA recall	WA support	roc_auc
4	XGBoost	0.74	0.74	0.75	0.74	174.0	0.78
3	Random Forest	0.73	0.72	0.75	0.72	174.0	0.77
7	CatBoost	0.73	0.72	0.74	0.72	174.0	0.79
9	AdaBoost	0.73	0.72	0.74	0.72	174.0	0.77
6	LightGBM	0.71	0.71	0.73	0.71	174.0	0.80
1	Decision Tree	0.70	0.70	0.70	0.70	174.0	0.69
8	HistGradientBoosting	0.69	0.69	0.71	0.69	174.0	0.78
5	Gradient Boosting	0.67	0.67	0.69	0.67	174.0	0.77
0	Logistic Regression	0.65	0.64	0.66	0.64	174.0	0.70
2	KNeighbors	0.62	0.63	0.62	0.63	174.0	0.67

лучшая модель с гиперпараметрами:

```
[17]: model = CatBoostClassifier(
    bootstrap_type='Bayesian',
    border_count=47,
    depth=6,
    feature_border_type='Uniform',
    grow_policy='SymmetricTree',
    iterations=59,
    l2_leaf_reg=1.92,
    leaf_estimation_iterations=7,
    learning_rate=0.17,
    random_strength=0.73,
    random_state=42,
    verbose=0,
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_score = model.predict_proba(X_test)[:, 1]

report = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report).transpose()
roc_auc = roc_auc_score(y_test, y_score)

print("Accuracy:", round(report['accuracy'], 2))
print("ROC AUC Score:", round(roc_auc, 2))
report_df
```

Accuracy: 0.71
ROC AUC Score: 0.78

```
[17]:
```

	precision	recall	f1-score	support
0	0.809524	0.666667	0.731183	102.000000
1	0.622222	0.777778	0.691358	72.000000
accuracy	0.712644	0.712644	0.712644	0.712644
macro avg	0.715873	0.722222	0.711270	174.000000
weighted avg	0.732020	0.712644	0.714704	174.000000

CC50 median

[11]:

	Model	WA f1	accuracy	WA precision	WA recall	WA support	roc_auc
2	KNeighbors	0.74	0.74	0.75	0.74	200.0	0.79
6	LightGBM	0.74	0.74	0.75	0.74	200.0	0.83
9	AdaBoost	0.74	0.74	0.75	0.74	200.0	0.81
3	Random Forest	0.73	0.73	0.73	0.73	200.0	0.84
4	XGBoost	0.73	0.73	0.74	0.73	200.0	0.83
5	Gradient Boosting	0.73	0.73	0.73	0.73	200.0	0.83
7	CatBoost	0.73	0.73	0.73	0.73	200.0	0.83
0	Logistic Regression	0.71	0.71	0.71	0.71	200.0	0.78
8	HistGradientBoosting	0.70	0.70	0.70	0.70	200.0	0.81
1	Decision Tree	0.69	0.69	0.70	0.69	200.0	0.69
10	Gaussian	0.68	0.69	0.71	0.69	200.0	0.71

лучшая модель с гиперпараметрами:

```
[14]: model = CatBoostClassifier(
        bootstrap_type='MVS',
        border_count=173,
        depth=9,
        feature_border_type='Uniform',
        grow_policy='SymmetricTree',
        iterations=256,
        l2_leaf_reg=5.27,
        leaf_estimation_iterations=3,
        learning_rate=0.02,
        random_strength=0.64,
        random_state=42,
        verbose=0,
    )
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    y_score = model.predict_proba(X_test)[:, 1]

    report = classification_report(y_test, y_pred, output_dict=True)
    report_df = pd.DataFrame(report).transpose()
    roc_auc = roc_auc_score(y_test, y_score)

    print("Accuracy:", round(report['accuracy'], 2))
    print("ROC AUC Score:", round(roc_auc, 2))
    report_df
```

Accuracy: 0.74
ROC AUC Score: 0.83

```
[14]:
```

	precision	recall	f1-score	support
0	0.742857	0.757282	0.750000	103.00
1	0.736842	0.721649	0.729167	97.00
accuracy	0.740000	0.740000	0.740000	0.74
macro avg	0.739850	0.739466	0.739583	200.00
weighted avg	0.739940	0.740000	0.739896	200.00

SI median

	Model	WA f1	accuracy	WA precision	WA recall	WA support	roc_auc
4	XGBoost	0.72	0.72	0.72	0.72	170.0	0.78
7	CatBoost	0.72	0.72	0.72	0.72	170.0	0.78
6	LightGBM	0.71	0.71	0.71	0.71	170.0	0.79
9	AdaBoost	0.69	0.69	0.69	0.69	170.0	0.79
5	Gradient Boosting	0.68	0.68	0.68	0.68	170.0	0.77
1	Decision Tree	0.67	0.67	0.67	0.67	170.0	0.70
3	Random Forest	0.67	0.67	0.67	0.67	170.0	0.77
8	HistGradientBoosting	0.67	0.67	0.67	0.67	170.0	0.77
2	KNeighbors	0.66	0.66	0.69	0.66	170.0	0.70
0	Logistic Regression	0.65	0.65	0.66	0.65	170.0	0.69

лучшая модель с гиперпараметрами:

```

: model = LGBMClassifier(
    verbose=-1,
    colsample_bytree=0.79,
    learning_rate=0.134,
    max_depth=3,
    min_child_samples=9,
    min_split_gain=0.006,
    n_estimators=55,
    num_leaves=47,
    reg_alpha=0.81,
    reg_lambda=1,
    subsample=1,
    random_state=42,
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_score = model.predict_proba(X_test)[:, 1]

report = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report).transpose()
roc_auc = roc_auc_score(y_test, y_score)

print("Accuracy:", round(report['accuracy'], 2))
print("ROC AUC Score:", round(roc_auc, 2))
report_df

```

Accuracy: 0.69
ROC AUC Score: 0.77

```

:

```

	precision	recall	f1-score	support
0	0.686047	0.702381	0.694118	84.000000
1	0.702381	0.686047	0.694118	86.000000
accuracy	0.694118	0.694118	0.694118	0.694118
macro avg	0.694214	0.694214	0.694118	170.000000
weighted avg	0.694310	0.694118	0.694118	170.000000

SI > 8

	Model	WA f1	accuracy	WA precision	WA recall	WA support	roc_auc
3	Random Forest	0.74	0.74	0.74	0.74	170.0	0.76
5	Gradient Boosting	0.74	0.74	0.74	0.74	170.0	0.79
7	CatBoost	0.74	0.75	0.75	0.75	170.0	0.78
4	XGBoost	0.73	0.73	0.73	0.73	170.0	0.80
8	HistGradientBoosting	0.73	0.74	0.74	0.74	170.0	0.77
2	KNeighbors	0.70	0.72	0.75	0.72	170.0	0.75
6	LightGBM	0.70	0.71	0.70	0.71	170.0	0.79
9	AdaBoost	0.70	0.70	0.70	0.70	170.0	0.75
0	Logistic Regression	0.68	0.68	0.68	0.68	170.0	0.71
1	Decision Tree	0.67	0.67	0.67	0.67	170.0	0.68
10	Gaussian	0.56	0.62	0.62	0.62	170.0	0.57

лучшая модель с гиперпараметрами:

```

split = 5, n_estimators = 100)
[44]: model = RandomForestClassifier(
        n_estimators=100,
        random_state=42,
    )
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    y_score = model.predict_proba(X_test)[:, 1]

    report = classification_report(y_test, y_pred, output_dict=True)
    report_df = pd.DataFrame(report).transpose()
    roc_auc = roc_auc_score(y_test, y_score)

    print("Accuracy:", round(report['accuracy'], 2))
    print("ROC AUC Score:", round(roc_auc, 2))
    report_df

```

Accuracy: 0.74
ROC AUC Score: 0.76

```
[44]:
```

	precision	recall	f1-score	support
0	0.745455	0.836735	0.788462	98.000000
1	0.733333	0.611111	0.666667	72.000000
accuracy	0.741176	0.741176	0.741176	0.741176
macro avg	0.739394	0.723923	0.727564	170.000000
weighted avg	0.740321	0.741176	0.736878	170.000000

Сравнение моделей

В каждом эксперименте запускались все модели и выводились в табличном виде для сравнения с сортировкой по ключевой метрике - r^2 для регрессии и weighted avg f1-score для классификации.

Обсуждение

Ограничение исследования

Исследование ограничено заданным датасетом.

Возможности для дальнейшего исследований

Обработку данных улучшила бы консультация с профессиональным химиком, чтобы вывести новые признаки и обработать текущие не общими методами. Из моделей можно попробовать использовать глубокое обучение.

Заключение

Выводы

Полученные модели, исходя из метрик, эффективны в поиске химического вещества для лекарственного препарата от гриппа.

Практическое значение

В мире современной химии затруднена цифровизация первичных данных. Нет единого формата для обмена информацией. Эти факторы усложняют использование полученных моделей, так как придется потратить время, чтобы собрать и преобразовать данные в тот формат признаков, на которых проводилось исследование. Поэтому данная работа не претендует на глобальное решение поиска эффективных химических веществ. Однако, как минимум на этом 1001 образце можно получить N(например, 10) самых эффективных, чтобы проверить их на практике, что уже сэкономит не мало ресурсов. Поэтому считаю, что данная работа будет полезна в локальных исследованиях, как составная часть сложного процесса по синтезу лекарственных препаратов.